SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Engineer to Excel

Accredited by NAAC A Grade

SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

ASSIGNMENT ANSWERS

1. Online Banking Fund Transfer: Equivalence Partitioning and Boundary Value Analysis

Introduction

An online banking system must process fund transfers accurately because even a small error can cause financial loss, customer complaints, and regulatory problems. The reported failures occur mainly at minimum and maximum transfer limits. This indicates that the existing test suite may concentrate on normal values and may not test boundary conditions systematically. Random testing can discover unexpected failures, but it is not sufficient as the main test design method.

Analysis of the existing strategy

If testers select only a few typical amounts, they may miss defects such as an incorrect less-than operator, an off-by-one error, a wrong limit configuration, or a rounding problem. For example, a requirement may state that a transfer from ₹100 to ₹50,000 is allowed. A program using amount > 100 instead of amount >= 100 would reject the minimum valid amount. Similarly, a program using amount < 50,000 instead of amount <= 50,000 would reject the maximum valid amount. Testing only ₹500 or ₹5,000 would not reveal these defects.

Equivalence Partitioning (ECP)

ECP divides input values into groups expected to behave similarly. Assuming the stated permitted range is ₹100–₹50,000, the classes are: (1) amount below ₹100—invalid, (2) amount from ₹100 through ₹50,000—valid, and (3) amount above ₹50,000—invalid. Additional classes should cover

zero, negative values, blank input, non-numeric characters, decimal precision, and values exceeding the supported numeric range. One representative from each class is selected, such as ₹50, ₹5,000, and ₹50,001. ECP reduces unnecessary duplication while maintaining meaningful input coverage.

Boundary Value Analysis (BVA)

Because defects cluster near limits, test values immediately below, at, and immediately above each boundary. For the lower limit, test ₹99, ₹100, and ₹101. For the upper limit, test ₹49,999, ₹50,000, and ₹50,001. Also test ₹0 and a negative amount where the system's validation rules require rejection. Verify both the displayed result and the actual account balances, transaction record, and error message. The expected result must be based on the approved business requirement, not assumptions.

Role of random testing

Random testing can be retained as a supplementary technique. It can generate varied amounts and sequences that a tester may not anticipate. However, random values may rarely land exactly on a boundary and do not guarantee requirement coverage. A reproducible random seed and clear failure logging are useful.

Recommended approach and conclusion

Use requirement-based test cases, ECP, BVA, invalid-input tests, and integration tests for account balance updates. Include insufficient funds, daily transfer limits, duplicate submissions, network interruption, and rollback scenarios. Automate repeatable boundary cases and run regression tests after fixes. This structured approach is more efficient and traceable than random testing alone, while random testing remains useful for discovering additional defects.

2. Healthcare Application: White-Box Testing and Coverage Adequacy

Introduction and problem analysis

A healthcare application may calculate dosage, validate patient details, check allergies, or decide whether a clinical action is permitted. Critical defects found after deployment suggest that the test suite did not adequately exercise important statements, decisions, or execution paths. In healthcare, a test report showing a high percentage is not enough unless the tested behaviour is relevant to patient safety.

Conceptual control flow graph

Consider a simplified medication-order function:

Start → Validate patient → Is patient valid?

If no, return validation error.

If yes, check allergy information → Is a serious allergy present?

If yes, block order and raise an alert.

If no, validate dosage → Is dosage within the permitted range?

If no, reject the order.

If yes, save the order → End.

Each processing step is a node, and each possible transfer of control is an edge. The graph must include every decision outcome, error handler, exception route, and relevant loop. Missing edges in the graph can lead to untested behaviour.

Coverage criteria

Statement coverage asks whether each executable statement has run at least once. It may miss a false outcome of a decision. Branch coverage requires each decision outcome, such as true and false, to be executed. It is stronger than statement coverage but does not guarantee that every combination of conditions is tested. Path coverage considers complete routes through the graph. Full path coverage is often impossible when loops create many or infinite paths, so critical paths and bounded loop behaviours must be selected.

Likely coverage gaps

The previous approach may have tested only valid patients, non-allergic patients, and normal dosages. It may have missed invalid patient identifiers, unavailable allergy records, allergy present, dosage below minimum, dosage above maximum, database failure, and simultaneous updates. It may also have ignored how the system behaves when a service is unavailable or data is incomplete.

Improved strategy

First, review requirements with clinical stakeholders and identify safety-critical decisions. Build a control flow graph from the implementation and map every node and branch to test cases. Aim for 100% statement and branch coverage in safety-critical decision logic where feasible, while documenting justified exclusions. Add condition coverage for compound Boolean expressions, exception-path testing, data validation, and integration testing with databases and external clinical services. Use boundary tests for dosage limits and equivalence classes for patient categories. Add negative tests for missing, stale, or contradictory data.

Conclusion

White-box coverage helps reveal unexecuted code, but coverage is not proof that the application is correct. Combine structural coverage with requirement-based expected results, risk-based prioritization, code review, regression automation, and clinical validation. This improves reliability and reduces the probability that dangerous conditions remain undetected.

3. Retail E-Commerce: Testing Price, Quantity, and Discount Combinations

Problem analysis

An e-commerce checkout calculation depends on interacting inputs: item price, quantity, and discount. Testing each input independently may not reveal a defect caused by a particular combination. For example, a discount might be applied before quantity multiplication when the requirement expects it after subtotal calculation, or a percentage discount may be mistakenly treated as a fixed amount. The test strategy must therefore include both individual input partitions and combinations of business rules.

Equivalence class partitioning

For price, define valid positive prices, zero price where permitted, negative price, missing value, and non-numeric value. For quantity, define valid positive whole numbers, zero, negative numbers, fractional quantities if not permitted, and values above the maximum order quantity. For discount, define no discount, valid percentage discount, maximum permitted discount, invalid negative discount, and discount greater than the allowed maximum. Select representative values from each class rather than testing every possible value.

Boundary value analysis

Test values immediately below, at, and above each limit. If the minimum price is ₹1 and the maximum allowed line-item price is ₹100,000, test ₹0.99 where decimals are accepted, ₹1, ₹1.01, ₹99,999, ₹100,000, and ₹100,001. If quantity is restricted to 1–100, test 0, 1, 2, 99, 100, and 101. If the maximum discount is 30%, test 29%, 30%, and 31%. The exact values should be aligned with the actual requirements and currency precision.

Decision table testing

A decision table records combinations of conditions and the expected action. Conditions may include: price valid, quantity valid, discount valid, customer eligible for promotion, and subtotal above a threshold. Actions include calculate total, apply discount, reject input, or show an error. A

sample rule could state that if price, quantity, and discount are valid and the customer qualifies, calculate subtotal and apply the permitted discount. If any input is invalid, reject the checkout calculation and display a specific validation message. The table helps expose conflicting or missing rules.

Other useful methods

Use pairwise testing when there are many factors and exhaustive testing is too expensive. Pairwise testing ensures every pair of factor values occurs at least once, but it does not replace testing of known high-risk three-way combinations. Add arithmetic checks, tax and shipping interactions, rounding cases, and currency precision. Compare the application total against an independently calculated expected result.

Revised test design and conclusion

Create traceable tests from requirements, use ECP and BVA for each field, then use decision tables and selected combination tests for interactions. Automate core checkout scenarios and repeat them as regression tests. Include invalid inputs and verify that failed calculations do not create orders or charge customers. This gives better defect detection and a more complete test suite without testing every theoretical combination.

4. Complex Nested Conditions and Loops: Path Testing Strategy

Evaluation of current testing

A module with nested conditions and loops can have many possible execution routes. Reporting partial path coverage without explaining which paths were tested makes it difficult to judge adequacy. Common gaps include an incomplete control flow graph, omitted false branches, incorrect loop limits, and failure to identify independent paths. If testers exercise only the normal route, errors in exceptional or repeated execution may remain hidden.

Control flow graph and independent paths

A control flow graph (CFG) represents program statements as nodes and control transfers as directed edges. Decision points have multiple outgoing edges. To plan basis path testing, identify the graph's nodes, edges, and connected components. For a single connected graph, cyclomatic complexity is commonly calculated as $V(G) = E - N + 2$, where E is the number of edges and N is the number of nodes. It can also be calculated as the number of decision nodes plus one for a

structured single-entry, single-exit graph. This gives the number of linearly independent paths in a basis set.

Nested condition example

Suppose a function validates a user, loops through submitted records, and checks whether each record meets a threshold. Test paths should include invalid user, valid user with no records, one valid record, one invalid record, a mixture of valid and invalid records, and an early exit caused by an error. For nested decisions, test both true and false outcomes at each decision. Where compound conditions exist, select cases that show the effect of individual conditions.

Loop boundary tests

For each loop, test zero iterations if allowed, one iteration, two iterations, a typical number of iterations, the maximum permitted number, and one above the maximum. For nested loops, vary the inner and outer loop counts independently. Test termination conditions carefully, because off-by-one errors may cause skipped records or infinite loops. Include empty collections and malformed data.

Proposed complete strategy

Draw and review the CFG against the source code. Calculate cyclomatic complexity and select a basis set of independent paths. Create at least one test for each selected path, and add loop-boundary and exception tests. Use branch coverage to ensure both outcomes of each decision are executed. Instrument tests to capture which paths and branches ran. If a path is infeasible, document why rather than silently excluding it.

Expected outcomes and conclusion

Path testing improves confidence that different control routes behave correctly. However, complete path coverage may be impractical for loops or complex systems. The practical goal is to cover independent paths, all decision outcomes, critical loop boundaries, and error-handling routes, prioritizing those with the greatest risk.

5. Financial Application: Integrated Black-Box and White-Box Testing

Introduction

A financial application can continue to produce production defects even when both black-box and white-box testing are reported. This often happens when the two approaches are performed separately, coverage percentages are treated as the only success measure, or test cases are not linked

to business requirements. Financial systems require accurate calculations, secure authorization, reliable transactions, and predictable recovery from failures.

Critical evaluation

Black-box testing checks observable behaviour without relying on internal code structure. It is useful for verifying transfer limits, fees, account rules, and error messages. White-box testing examines statements, branches, conditions, and paths in the implementation. It can reveal unexecuted branches and faulty control logic. Neither method alone is sufficient: black-box tests may miss internal paths, while white-box tests may execute code without checking whether the output is correct according to business requirements.

Combined test design

Begin with a requirements and risk review. Identify critical functions such as deposits, withdrawals, transfers, interest calculations, fees, refunds, and account permissions. For each input, apply equivalence class partitioning and boundary value analysis. For example, test the minimum and maximum transaction amount, values just below and above them, insufficient balances, and invalid account numbers. Use decision tables for rules involving account type, transaction type, user role, and daily limits.

Next, create CFGs for complex calculation and authorization functions. Measure statement and branch coverage, and use basis path testing for important control logic. For compound Boolean expressions, add condition coverage or MC/DC where the risk and standards justify it. Review exception paths such as database timeout, service interruption, duplicate request, and failed rollback.

Test adequacy and metrics

Coverage metrics should be interpreted alongside requirements coverage, defect detection rate, severity, and risk. A suite with 100% statement coverage may not test all branches or validate expected financial results. Each test should have a requirement identifier, input data, expected result, actual result, and execution status. Any uncovered code should be investigated and documented.

Improved hybrid workflow

Use unit tests for calculation and decision logic, integration tests for databases and payment services, system tests for end-to-end user journeys, and regression tests after every fix. Include

concurrency tests, idempotency tests, audit-log checks, access-control tests, and reconciliation of account balances. Use code review and static analysis to detect issues before execution.

Conclusion

A hybrid strategy is effective when black-box tests verify the business contract and white-box tests expose structural gaps. Traceability, risk-based prioritization, meaningful coverage targets, and automated regression testing help prevent recurring defects and improve financial correctness.

6. HR Payroll Bonus Errors at 5, 10, and 15 Years of Service

Scenario analysis

The HR payroll system calculates monthly bonuses using years of service and performance ratings. Incorrect payouts for employees at exactly 5, 10, and 15 years strongly suggest boundary-related defects. The implementation may use incorrect comparison operators, overlapping service bands, inconsistent rounding, or a mismatch between the written policy and the coded rules. Because payroll affects employee compensation, tests must use explicit expected values approved by HR.

Equivalence classes

Assume, only for illustrating the test design, that the policy has service bands of less than 5 years, 5 to less than 10 years, 10 to less than 15 years, and 15 years or more. The real payroll policy must replace this example if its rules differ. These bands form equivalence classes because employees in each band are expected to follow the same service-related bonus rule. Performance ratings should also be partitioned into all permitted categories, such as low, meets expectations, and exceeds expectations, according to the actual rating scale. Invalid or missing ratings should form separate classes.

Boundary value analysis

For the 5-year boundary, test 4 years 11 months, exactly 5 years, and 5 years 1 month. For the 10-year boundary, test 9 years 11 months, exactly 10 years, and 10 years 1 month. For the 15-year boundary, test 14 years 11 months, exactly 15 years, and 15 years 1 month. If service is calculated by exact dates, test the day before the anniversary, the anniversary date, and the day after. Confirm whether service is measured in completed years or rounded years. Test each boundary with more than one performance rating to detect interaction defects.

Expected-result validation

Build a decision table mapping service band and performance rating to the bonus percentage or

amount. For every test, calculate the expected bonus independently and compare it with the application result. Check currency rounding, part-time status if relevant, unpaid leave rules, and employees whose hire date or rating is missing. Verify that the correct payroll period is used.

Why structured testing is preferable to exploratory testing

Exploratory testing can reveal unexpected behaviour, but it does not guarantee that exact threshold values are tested consistently. ECP reduces redundant cases, while BVA deliberately focuses on likely defect locations. The methods are repeatable, auditable, and suitable for regression testing.

Conclusion

Use a policy-approved decision table, ECP for service and rating categories, BVA around every threshold, and automated regression tests. Include negative tests and verify generated payslips and payroll totals. This systematic design is more reliable than relying on exploratory testing alone.

7. Laboratory Temperature Control: CFG and MC/DC

Safety context

A laboratory temperature controller may switch a heater or cooler on and raise an alarm when sensor readings are invalid. A missed alarm after a sensor-malfunction sequence is a serious defect. Testing must cover the control flow, each decision outcome, individual Boolean conditions, and fault-handling behaviour. Test thresholds and alarm rules must come from the system's approved safety specification.

Illustrative control logic

Consider these conditions: sensor_valid, temperature_high, temperature_low, and backup_sensor_valid. A simplified flow is: Start → read sensors → check sensor validity. If the primary sensor is invalid, check the backup sensor. If both sensors are invalid, activate the alarm and place the controller in a safe state. If readings are valid, compare temperature against the upper and lower thresholds. If temperature is too high, turn the cooler on; if too low, turn the heater on; otherwise keep both off. Then update the status and end the cycle. A CFG should show every decision and each true/false edge.

Branch coverage gap

Testing only normal temperature values may execute the main statements but fail to test sensor-invalid branches. Even 100% branch coverage does not always show that each condition independently affects a compound decision. For example, a decision such as “sensor invalid OR

temperature dangerously high" may evaluate true because of either condition, but tests must demonstrate the independent effect of each condition.

MC/DC approach

Modified Condition/Decision Coverage requires each atomic condition in a decision to be shown to independently affect the decision outcome while the other conditions are held appropriately constant. For a decision A OR B, use test pairs such as A=false, B=false → false; A=true, B=false → true; and A=false, B=true → true. The first and second pair demonstrates A's independent effect, while the first and third demonstrates B's independent effect. For more complex decisions, construct a condition table and identify pairs that isolate each condition. Actual controller logic may require additional cases for valid sensors, high/low thresholds, and backup status.

Test scenarios

Test normal temperature, temperature exactly at each threshold, just below and above thresholds, primary sensor failure, backup sensor failure, both sensors failing, contradictory readings, delayed sensor data, and recovery after a fault. Verify the alarm signal, heater/cooler outputs, logged event, and safe-state transition. Simulate sequences rather than testing only isolated readings.

Conclusion

A strong white-box strategy uses CFG review, branch and condition coverage, MC/DC for safety-critical Boolean decisions, and boundary testing for temperatures. Hardware-in-the-loop tests should verify that software commands correctly operate real sensors, alarms, heaters, and coolers. MC/DC improves confidence in decision logic but must be combined with requirements validation and physical safety tests.

8. Travel Booking Portal: Promo-Code Combinations and Negative Totals

Problem analysis

A travel portal may accept promotional codes based on ticket price, promotion type, and user tier. Testing each input separately does not reveal defects caused by interactions—for example, applying two percentage discounts in the wrong order, combining a fixed discount with a percentage discount, or allowing discounts to exceed the eligible amount. A negative total is a financial correctness defect and should never be accepted unless the business rules explicitly define a credit mechanism.

Decision table design

List conditions such as: base price is valid; promotion code is valid; promotion is compatible with another code; user tier is eligible; minimum purchase threshold is met; and discount is capped at the permitted amount. Define actions such as apply discount, reject code, display an explanation, calculate tax, or prevent checkout. Each column in the decision table represents a rule. For example, a valid price plus an eligible code and qualifying user tier may permit a discount. An expired code should be rejected. Two mutually exclusive codes should not both be applied. If a discount would exceed the payable amount, the system should apply the documented cap or reject the combination.

Cause-effect graphing

Identify causes (valid price, code active, tier eligible, code compatible) and effects (discount accepted, total recalculated, checkout allowed, error displayed). Connect them with logical relationships and constraints. This helps identify missing combinations and contradictory rules before generating tests.

Optimized test set

Include a normal booking with no code, one valid fixed discount, one valid percentage discount, an expired code, a code for the wrong user tier, a price below the minimum purchase threshold, two compatible codes if allowed, two incompatible codes, a maximum discount, and a combination whose raw discount exceeds the base price. Test zero and negative prices as invalid input. Also test decimal rounding, taxes, fees, cancellation/refund calculations, and the effect of removing a code after it has been applied.

Expected-result checks

For each scenario, calculate the expected total independently. Confirm that the total cannot become negative, the order summary updates immediately, and the backend—not only the user interface—enforces the rules. Re-submit requests and refresh the page to check that a discount is not applied twice. Verify audit records and error messages.

Minimizing redundancy

Equivalence classes and decision tables reduce duplicate cases by grouping equivalent situations. Pairwise coverage can help when there are many factors, but high-risk combinations should be tested explicitly rather than left to pairwise selection alone. Keep a traceability matrix from each promotional rule to its tests.

Conclusion

Decision table testing and cause-effect graphing are appropriate because the defect depends on interactions. They systematically represent valid and invalid combinations, improve coverage, and keep the test suite smaller than exhaustive combination testing.

9. Legacy Database Migration: Infinite Loops Despite High Statement Coverage

Why statement coverage was insufficient

High statement coverage means that executable statements have run, but it does not prove that loops terminate correctly or that their boundary conditions are handled. A migration tool may execute every statement during a normal run and still enter an infinite loop when a record is malformed, a cursor does not advance, a retry counter is not updated, or a termination condition uses the wrong comparison operator. The reported production crashes therefore indicate that the team needs stronger structural and loop testing.

Control flow and cyclomatic complexity

Construct a CFG for the migration routine, including nested loops, validation decisions, retry branches, exception handlers, and database cursor movement. Calculate cyclomatic complexity using $V(G) = E - N + 2P$, where E is the number of edges, N the number of nodes, and P the number of connected components. For a single connected routine, P is 1. The result estimates the number of linearly independent paths required for basis path testing. It is a planning measure, not a guarantee that every possible execution sequence has been covered.

Basis path strategy

Identify a basis set of independent paths and select test data to execute each. Include a valid record, invalid record, empty input table, one-record table, mixed records, duplicate records, database read failure, write failure, and recovery after a failed record. Ensure tests cover both success and error branches and verify that the migration either completes or fails safely with a clear report.

Loop boundary testing

For each loop, test zero iterations, one iteration, two iterations, a typical volume, the configured maximum, and a value just above the maximum if a limit exists. For nested loops, test different outer and inner counts, including empty inner collections. Check that the loop variable, cursor, or iterator advances on every continuing path. Test malformed records that trigger retry logic and

ensure retries are bounded. A timeout can act as a safety guard in automated tests, but it does not replace finding the logical reason for non-termination.

Data integrity checks

After migration, compare source and destination record counts, keys, required fields, and selected checksums or aggregate totals. Test rollback or restart behaviour after interruption. Confirm that rerunning the migration does not create duplicates or corrupt partially migrated data.

Conclusion

Use CFG-based basis path testing, cyclomatic complexity, loop-boundary tests, and data-integrity validation. Statement coverage should be retained as one metric, but it must be supplemented by branch/path analysis and termination assertions. These measures directly address the type of failure reported.

10. Food Delivery Tracker: State Transition Testing

Problem analysis

A real-time food delivery tracker moves through states such as Ordered, Preparing, Out for Delivery, and Delivered. A driver cancellation during delivery introduces an exceptional transition that may not be covered by tests of each state in isolation. The application can become stuck if the cancellation event is not handled, if the order status is not synchronized between services, or if the user interface expects only the normal sequence.

State model

Define states including Order Placed, Accepted, Preparing, Ready for Pickup, Out for Delivery, Delivered, Cancelled, and Exception/Support Required. Events include restaurant acceptance, food preparation completion, driver assignment, pickup, delivery confirmation, driver cancellation, customer cancellation, payment failure, and network recovery. Each transition should specify the triggering event, guard conditions, actions, and resulting state. For example, a driver cancellation while Out for Delivery may transition to Driver Reassignment or Support Required; the exact expected behaviour must follow product requirements.

Test cases

Test the normal sequence from Ordered to Delivered. Test cancellation before preparation, during preparation, after pickup, and after the driver has started delivery. Verify that a driver cancellation triggers reassignment or a clearly defined exception route rather than leaving the order indefinitely

Out for Delivery. Test repeated cancellation events, delayed notifications, duplicate event delivery, offline devices, and a customer refreshing the tracker. Check that payment/refund status, estimated arrival time, and customer notifications remain consistent.

Why ECP and BVA are insufficient

ECP and BVA are useful for individual input values and numeric boundaries, but they do not systematically test sequences of events. A tracker can accept every individual input correctly yet fail when events occur in an unexpected order. State transition testing checks both valid and invalid transitions and can reveal dead ends, missing transitions, and incorrect state updates.

Improved strategy

Create a state transition table and diagram. Ensure every state is entered, every valid transition is exercised, and important invalid transitions are rejected or handled safely. Include transition-pair testing, which checks consecutive pairs of transitions, and risk-based sequences around cancellation and payment. Add integration tests for the backend event service and mobile notifications. Use automated regression tests for all previously defective transitions.

Conclusion

State transition testing is the most suitable central technique for this dynamic workflow. Combined with ECP, BVA for time and price fields, and integration testing, it helps ensure that the tracker never becomes stuck and that users receive accurate order status information.

11. Cybersecurity Authentication: Hybrid Testing with Error Guessing and Path Analysis

Security problem

A multi-factor authentication module evaluates password strength, OTP timing, and IP location. Brute-force vulnerabilities may arise if repeated attempts are not limited, OTPs can be reused, expiry is not enforced, or different checks are evaluated in an unsafe order. Testing only successful login and a few incorrect passwords is inadequate because attackers deliberately explore unusual sequences and edge cases.

Black-box testing and error guessing

Black-box tests should cover valid and invalid passwords, correct and incorrect OTPs, expired OTPs, reused OTPs, missing fields, excessive attempts, locked accounts, unusual IP changes, and rate-limit responses. Error guessing adds security-focused scenarios based on likely implementation

mistakes: sending many OTP requests, submitting the same OTP repeatedly, racing simultaneous login requests, changing device or IP between steps, using an OTP after password reset, and retrying after a lockout. Testing must be performed only in an authorized test environment.

White-box path analysis

Review the control flow for password verification, account status, OTP generation and validation, expiry checks, IP risk evaluation, attempt counters, lockout logic, and session creation. Draw a CFG and identify independent paths through successful authentication, wrong password, wrong OTP, expired OTP, suspicious IP, locked account, and service failure. Verify that session creation is reachable only after all required checks pass. Check that error paths do not accidentally bypass rate limiting or reset counters incorrectly.

Test adequacy

Statement coverage alone is insufficient because the vulnerable behaviour may depend on a particular branch sequence. Branch coverage should exercise both outcomes of security decisions. Path testing should focus on high-risk routes, especially attempts that combine valid credentials with expired OTPs or suspicious locations. Review condition logic and test concurrent requests, as race conditions may not appear in sequential tests.

Security-specific expectations

Enforce server-side rate limits, short OTP validity, one-time use, secure randomness, account/device throttling, safe session handling, and monitoring. Avoid revealing whether the password or account name was the incorrect factor. Store audit events without logging secrets. Verify that lockout and recovery processes cannot be abused to deny service to other users. Use automated security tests and independent review.

Conclusion

Combining error guessing with formal path analysis offers broader coverage than either method alone. Error guessing explores realistic attacker behaviour, while CFG and path analysis expose missing structural checks. Security testing should also include threat modelling, dependency checks, secure code review, and authorized penetration testing. No single coverage percentage proves that an authentication module is secure.

12. E-Learning Subscription Upgrade on Expiration Date

Scenario analysis

An e-learning platform grants Free, Basic, and Premium access based on subscription dates and course types. Errors when users upgrade exactly on the expiration date suggest ambiguity about date boundaries, time zones, inclusive/exclusive rules, and the order in which expiry and upgrade events are processed. A user may lose access early, receive an unintended extra day, or be charged without receiving the correct access.

Equivalence partitioning for dates

Partition date/time inputs into: before expiration, exactly at the expiration boundary, after expiration, future start date, invalid date, and missing date. Also consider active, expired, and scheduled subscriptions. For subscription types, create classes for Free, Basic, and Premium, and for each course type create classes that are included, excluded, or available only at a higher tier. These classes help select representative cases while preserving important business distinctions.

Boundary-focused examples

If a subscription expires at 23:59:59 on a specified date, test immediately before, exactly at, and immediately after that instant. The system requirements must clearly define whether the expiration timestamp is inclusive or exclusive. Test upgrades initiated before expiry but confirmed after expiry, upgrades at midnight, leap-day dates, month-end dates, daylight-saving transitions where relevant, and time-zone conversion between the server and user. Use the platform's actual policy for the expected outcome.

Decision table and scenarios

Create a table using current tier, requested tier, current subscription status, current time relative to expiration, payment result, and course access rule. Actions should specify whether payment is charged, access is granted, the expiry date is extended, and a confirmation is sent. Test Free-to-Basic, Basic-to-Premium, Premium renewal, failed payment, expired subscription, and an upgrade exactly at the boundary. Verify that failed payment does not grant paid access and successful payment does not create duplicate charges.

Integration and regression testing

Check the payment gateway, subscription database, course authorization service, and notification service together. Confirm that the displayed plan, stored plan, payment record, and actual course

access agree. Test repeated upgrade clicks and delayed callbacks to prevent duplicate processing. Use a fixed test clock where possible to make boundary tests reproducible.

Conclusion

ECP organizes date and subscription categories, while BVA targets exact expiry boundaries. Decision tables cover interactions among tier, date, payment, and course type. A clearly defined time policy and automated regression tests make transitions more reliable and reduce disputes over access and billing.

13. Smart-Home Lighting: Branch and Condition Coverage

Risk of statement-only testing

A smart-home lighting controller uses switch-case statements and Boolean expressions to adjust brightness based on ambient light sensors. Statement coverage can show that each statement ran but may not demonstrate that every switch case or logical condition has been evaluated in both relevant ways. For example, a test may execute the dimming statement because the room is dark but never test sensor failure, manual override, or a contradictory sensor reading.

CFG and decision analysis

Construct a CFG showing sensor reading, validation, manual override, ambient-light categories, brightness calculation, and output to the lighting hardware. For each switch statement, test every case and the default case. For each decision, identify true and false outcomes. If the code uses conditions such as “sensor_valid AND (ambient_light < threshold OR manual_mode)”, test each atomic condition and determine whether changing it independently changes the overall decision.

Branch and condition coverage

Branch coverage ensures every decision outcome is executed. Condition coverage ensures each atomic Boolean condition takes both true and false values at least once. These are different measures: a compound decision can achieve branch coverage without exercising every condition independently. Where the logic is safety-critical or highly consequential, MC/DC may be appropriate. Tests should cover low, medium, and high ambient light, threshold values, manual override enabled/disabled, sensor valid/invalid, and combinations of these conditions.

Integration risks

Software may calculate the correct brightness but send an incorrect command to a hardware driver. The sensor may return stale data, use an unexpected unit, or disconnect intermittently. Therefore,

test sensor interfaces, output commands, timing, and recovery behaviour. Include tests for startup, power cycling, network loss, and manual control overriding automatic control.

Improved strategy

Use requirements-based tests for each lighting mode, branch and condition coverage for Boolean logic, BVA around light thresholds, and integration tests with simulated or actual hardware. Confirm expected brightness values independently and verify safe defaults when sensors fail. Use code review and static analysis to detect missing default cases and unreachable logic. Automate regression tests after code changes.

Conclusion

Statement coverage alone is too weak for complex decision logic. Branch and condition coverage reveal more gaps, while CFG review makes control routes explicit. Combining structural tests with hardware-software integration tests reduces the risk of incorrect lighting behaviour in real homes.

14. Logistics Shipping Costs: Robust Boundary Value Analysis

Problem analysis

A logistics system calculates shipping costs from package weight, destination zone, and delivery speed. The team missed zero and negative weights, showing that input validation focused on normal positive values. Such invalid inputs may produce free shipping, negative charges, arithmetic errors, or inconsistent invoices. Negative scenarios are necessary because users, external systems, or corrupted data can supply values outside the intended range.

Robust BVA

Assume the minimum valid weight is greater than zero and the system has a configured maximum weight Wmax. Robust boundary tests include a negative weight, zero, the smallest valid positive weight, a value just above that minimum, the maximum permitted weight, and a value just above the maximum. If weight is accepted to two decimal places, tests should reflect the smallest supported increment, such as 0.01 kg, rather than an arbitrary decimal. The actual limits must come from the shipping specification.

Equivalence classes and other factors

Partition weight into negative, zero, valid range, above maximum, missing, non-numeric, and excessive precision. Partition destination into supported zones and unsupported/unknown zones. Partition delivery speed into available services and invalid options. Test combinations such as valid

weight with invalid destination, zero weight with express delivery, and overweight package with an unsupported service. This checks that one valid field does not hide another invalid field.

Expected results

For invalid weight, the system should reject the calculation and display a clear message rather than silently charging zero or a negative amount. For valid weight, verify the price against the approved rate table, including zone surcharges, express fees, fuel surcharges, taxes, and rounding. Test weight conversion between grams and kilograms and ensure that units are not mixed. If fractional weight is rounded up for billing, confirm that behaviour explicitly.

Negative and robustness testing

Negative tests assess how the system behaves when input violates requirements. They help verify graceful failure, clear errors, and protection against invalid database records. Also test extremely large numeric values, blank fields, whitespace, alphabetic characters, and malformed API requests. Validate on both client and server sides, because client-side checks can be bypassed.

Conclusion

Robust BVA focuses on valid and invalid boundaries, while ECP covers the broader input classes. Combining them with destination and delivery-speed combinations gives a more complete test design. It prevents incorrect charges and makes the shipping calculator more robust against malformed data.

15. Medical Imaging: Basis Path Testing for Prime-Sized Datasets

Scenario analysis

A medical imaging module processes large datasets through five parallel processing loops. Results become inconsistent only when the data size is a prime number. This pattern may indicate an assumption about divisibility, incorrect chunk sizing, an indexing defect, a remainder that is not processed, a race condition, or a parallel worker receiving an empty or uneven workload. Random testing alone may not repeatedly select prime-sized datasets or make the failure easy to reproduce.

White-box investigation

Review how the dataset is divided among loops or worker threads. Check whether the code assumes the total number of elements can be evenly divided by the number of workers. Prime numbers are not divisible by many chunk sizes, so remainder handling is especially important. Inspect array bounds, start and end indices, synchronization, shared variables, and the merge step that combines

partial results. Construct a CFG for the key processing function, including validation, chunk allocation, loop exits, exception handling, and result aggregation.

Basis path testing and complexity

Calculate cyclomatic complexity for the relevant control flow graph using $V(G) = E - N + 2P$. Select a basis set of linearly independent paths and create inputs that execute each path. Include valid data, empty data, small data, unevenly divisible sizes, invalid dimensions, and failure in one worker. Basis path testing gives a structured way to exercise independent control routes, but it must be supplemented by data-size and concurrency tests.

Dataset-size test design

Test sizes such as 1, 2, 3, 5, 7, 11, and other representative prime numbers, as well as nearby composite sizes such as 4, 6, 8, 9, 10, and 12. Include sizes smaller than the number of workers, exactly equal to it, and larger sizes that do and do not divide evenly across workers. The particular sizes should be increased to production-scale volumes. Verify that every input element is processed exactly once and that no element is omitted or duplicated.

Mathematical accuracy and stability

Use a trusted serial implementation or independently calculated reference output to compare results. Check numerical tolerances, deterministic output where expected, memory use, thread completion, and exception handling. Repeat tests to reveal nondeterministic race conditions. Add property-based assertions such as output dimensions matching the specification and aggregate counts remaining consistent.

Conclusion

Basis path testing improves structural coverage, while targeted prime/composite dataset sizes investigate the observed trigger. Combine CFG analysis with loop-boundary tests, concurrency testing, and reference-result comparison. This provides stronger evidence of mathematical accuracy and system stability than random testing alone.

Code of Conduct

I **Sadhana/192421137** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Sadhana K***

Engineer to Excel

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification